1. Fundamentals of cloud computing by uploading and retrieving files from AWS S3 using a Python script

Aim:

To demonstrate the fundamentals of cloud computing by uploading and retrieving files from AWS S3 using a Python script.

Procedure:

1. Set up an AWS S3 Bucket with necessary permissions.
2. Create an IAM User with S3 access and obtain access keys.
3. Use the AWS SDK (Boto3) in a Python script to upload and download files.
4. Verify file upload and retrieval from the S3 bucket.

PROGRAM:

```
import boto3
import os
# S3 Configuration
bucket_name = 'karthikeyan-81883177059'
local_file = 'bmi.csv'
s3_key = 'uploaded-kk.txt'
# Create S3 client (credentials loaded securely)
s3 = boto3.client( 's3', aws_access_key_id='AKIA35JRHIKRRR6PLWZV',
        aws_secret_access_key='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',
        region_name='us-east-1'
        )
def upload_file():
    s3.upload_file(local_file, bucket_name, s3_key)
    print(f"{local_file} uploaded to s3://{bucket_name}/{s3_key}")
def download_file():
    download_path = f"downloaded-{local_file}"
    s3.download_file(bucket_name, s3_key, download_path)
    print(f"File downloaded as {download_path}")
upload_file()
download_file()
```

Result:

Successfully demonstrated cloud storage using AWS S3, highlighting cloud computing's flexibility and scalability.

---

2 AWS RDS MySQL Database

Aim

To demonstrate cloud database management by connecting a Python application to an AWS RDS MySQL instance to perform table creation, CSV data migration, and SQL querying.

Procedure

1. **Provision Instance:** Launch an AWS RDS MySQL instance (testdb) and configure Security Groups to allow inbound traffic on **port 3306**.
2. **Setup Environment:** Install pandas, sqlalchemy, and mysql-connector-python libraries in the Python environment.
3. **Establish Connection:** Define RDS credentials and use **SQLAlchemy** to create a database engine for cloud connectivity.
4. **Schema Creation:** Execute SQL commands to create a users table and verify its existence using SHOW TABLES.
5. **Data Processing:** Load the StudentsPerformance_Dataset.csv into a Pandas DataFrame for cleaning and preparation.
6. **Data Migration:** Use the to_sql() method to upload DataFrame records into the students_performance table in RDS.
7. **Verification:** Execute **SELECT** queries to retrieve and display records, ensuring the data was correctly inserted into the cloud.
8. **Connectivity Test:** Run a SELECT 1 heartbeat query to confirm a stable and active connection to the AWS instance.

Program

```
import pandas as pd
```

```
from sqlalchemy import create_engine

db_host = "database-1.c89agc4261l8.us-east-1.rds.amazonaws.com"

db_name = "testdb"        # ✅ correct database

db_user = "admin"

db_pass = "Datas"         # your password

engine = create_engine(

    f"mysql+mysqlconnector://admin:Datascience25@database-1.c89agc4261l8.us-east-1.rds.amazonaws.com:3306/testdb")

print("CONNECTED TO THE DATABASE")

#######################

import pandas as pd

from sqlalchemy import create_engine, text

#######################

create_table_query = text("""

CREATE TABLE IF NOT EXISTS users (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(100),

    email VARCHAR(100),

    age INT

        )

        """)

with engine.connect() as conn:

    conn.execute(create_table_query)

    conn.commit()   # IMPORTANT

print("Table created successfully")

#################################

pd.read_sql("SHOW TABLES", engine)

####################################

csv_path = "D:\AI&ML\StudentsPerformance_Dataset.csv"

df = pd.read_csv(csv_path)

df.head()

#################
```

```
df.to_sql(
    name="users",
    con=engine,
    if_exists="append",   # append data
    index=False
)
print("Data uploaded successfully")
##########
df.columns
############
df.to_sql(
    name="students_performance",
    con=engine,
    if_exists="replace",   # replace table if already exists
    index=False
)
print("CSV uploaded successfully into MySQL")
pd.read_sql("SELECT * FROM students_performance LIMIT 10", engine)
pd.read_sql("SHOW TABLES", engine)
#########
from sqlalchemy import create_engine, text
engine = create_engine(
    "mysql+mysqlconnector://admin:Datascience25@database-1.c89agc4261l8.us-east-1.rds.amazonaws.com:3306/testdb"
)
with engine.connect() as conn:
    result = conn.execute(text("SELECT 1"))
    print("AWS RDS CONNECTED SUCCESSFULLY")
```

Result

The Python application successfully established a secure connection to the AWS RDS MySQL database. Data was migrated from a CSV file to the cloud and verified through SQL queries, confirming functional cloud-based storage.

---

3. implimentation of contarinerization using docker for Python Application

Aim:

To create a Docker image for a Python application, run it as a container, and understand basic Docker commands. **Procedure:**

1. Docker installation is verified to ensure the Docker service is running properly.
2. A project directory is created and required files such as the Python program and Dockerfile are prepared.
3. A Docker image is built using the Dockerfile to package the application and its dependencies.
4. The built Docker image is verified by listing available images.
5. A Docker container is run from the image to execute the Python application.
6. The application output is observed to confirm successful execution.

Requirements:

Create a folder and put the following files in it.

1. **app.py :**

flask import Flask app = Flask(__name__) from @app.route('/') def home():

return "Hello from Flask running inside Docker!" if __name__ == "__main__": app.run(host="0.0.0.0", port=5000)

2. **requirements.txt** : flask
3. **Dockerfile (**without extension**) :**

FROM python:3.10-slim WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

To build the Docker image:

1. Open command prompt or terminal
2. Change the directory to the folder where the application stored (using “cd” command)
3. And run the command:

docker build -t flask-docker-app .

This will build the docker image based on Dockerfile

Steps to Run Image in Docker Desktop:

1. Once image is created, Open Docker Desktop and ensure the Docker engine is running.
2. Navigate to the Images section to view the built Docker image.
3. Click Run on the required image.
4. Configure container settings such as port mapping (e.g., 5000:5000).
5. Start the container and verify it in the Containers tab.
6. Access the application using http://localhost:5000.

Output :

Hello from Flask running inside Docker!

Result:

The Docker image was successfully created and the Python application was executed using a Docker container.

.

4.Implementation of SOAP Web Services in C#/JAVA Applications\

AIM:

To create and publish a SOAP-based web service using Java (JAX-WS) and to consume the same service using a Python client by sending and receiving SOAP XML messages,
thereby demonstrating platform-independent communication.

PROCEDURE:

Step 1

A Java class is created and annotated with @WebService to define a SOAP web service. The target namespace is specified to uniquely identify the service.

Step 2

Business logic methods such as add() and subtract() are defined using the @WebMethod annotation to make them accessible as SOAP operations.

Step 3

The web service is published using Endpoint.publish() by specifying a local URL. This starts an embedded HTTP server and generates the WSDL automatically.

Step 4

The web service is executed, and its availability is verified by accessing the WSDL through a web browser.

Step 5

A Python program is written to act as a SOAP client. A SOAP request message is manually constructed using XML format according to the WSDL specification.

Step 6

The SOAP request is sent to the Java web service using an HTTP POST request with appropriate headers.

Step 7

The Java SOAP service processes the request, executes the required method, and returns the result as a SOAP XML response.

Step 8

The SOAP response received in Python is parsed using XML parsing techniques, and the result is displayed.

Program:

```
//File name : CalculatorServices.java
 import javax.jws.WebMethod;
 import javax.jws.WebService;
 import javax.xml.ws.Endpoint;
   @WebService(     targetNamespace = "http://calculator.example.com/"
)
public class CalculatorService {
   @WebMethod
   public int add(int a, int b) {        return a + b;
   }
   @WebMethod
   public int subtract(int a, int b) {        return a-b;
   }
   public static void main(String[] args) {
      Endpoint.publish(
```

```
        "http://localhost:8080/calculator",            new CalculatorService()
      );
      System.out.println("SOAP Web Service is running...");
System.out.println("SOAP Web Service is running at http://localhost:8080/calculator");
   }
}
```

Next file name : requests.py

```
import requests ;
import xml.etree.ElementTree as ET ;


url = "http://localhost:8080/calculator" headers = {"Content-Type": "text/xml; charset=utf-8"} body ="""<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"                xmlns:cal="http://calculator.example.com/">
  <soap:Body>
    <cal:subtract>
      <arg0>10</arg0>
      <arg1>3</arg1>
    </cal:subtract>
  </soap:Body> </soap:Envelope>""" r = requests.post(url, data=body, headers=headers) r.raise_for_status() root = ET.fromstring(r.content) result = None namespaces = {
    "soap": "http://schemas.xmlsoap.org/soap/envelope/",
    "cal": "http://calculator.example.com/"
}
result = root.find(".//cal:subtractResponse/return", namespaces) if result is not None:
    print("value:", result.text) else:
    print("Result not found")


print("RAW SOAP RESPONSE:\n")
print(r.text)
```

RESULT:

The SOAP web service was successfully implemented using Java and consumed using Python. The service correctly processed the client request and returned the expected result, proving effective communication between heterogeneous systems using SOAP.

Output:

Access the service at: http://localhost:8080/calculator?wsdl

Client requests can call methods like add and subtract using SOAP.

5.Google App Engine using Render

Aim

To design and deploy a lightweight web application using the Flask framework that implements user authentication and session management. The project aims to provide a secure portal that displays dynamic cloud-related facts only to authorized users.

Procedure

1. **Initialize Application**: Setup the Flask environment and define a secret_key to track secure user sessions.
2. **Define Templates**: Create LOGIN_HTML and PORTAL_HTML strings with embedded CSS for a clean, centered interface.
3. **Handle Authentication**: Create a login route (/) that validates user input against pre-defined credentials (admin/123).
4. **Protect Portal**: Implement a /portal route that checks for an active session, redirecting unauthorized users to the login page.
5. **Serve Dynamic Content**: Use the random library to pick and display a different cloud fact upon every page refresh.
6. **Manage Sessions**: Develop a /logout route to clear session data and securely redirect the user back to the home screen.

Program

```
from flask import Flask, render_template_string, request, redirect, session
import random, os

app = Flask(__name__)
app.secret_key = "easy_key"

# Simple Login Page
LOGIN_HTML = """
<body style="text-align:center; padding:50px; font-family:sans-serif;">
```

```
    <h1>Cloud Login</h1>
    <form method="POST">
        <input type="text" name="u" placeholder="User" required><br><br>
        <input type="password" name="p" placeholder="Pass" required><br><br>
        <button type="submit">Login</button>
    </form>
    <p style="color:red">{{ err }}</p>
</body>
"""

# Simple Portal Page
PORTAL_HTML = """
<body style="text-align:center; padding:50px; font-family:sans-serif;">
    <h1>Cloud Portal</h1>
    <p><strong>Fact:</strong> {{ fact }}</p>
    <br>
    <a href="/portal">New Fact</a> | <a href="/logout">Logout</a>
</body>
"""

@app.route('/', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        if request.form['u'] == 'admin' and request.form['p'] == '123':
            session['user'] = 'admin'
            return redirect('/portal')
        return render_template_string(LOGIN_HTML, err="Try Again!")
    return render_template_string(LOGIN_HTML, err="")

@app.route('/portal')
def portal():
    if 'user' not in session: return redirect('/')
    facts = ["Cloud is scalable", "Cloud is cheap", "Cloud is fast"]
    return render_template_string(PORTAL_HTML, fact=random.choice(facts))

@app.route('/logout')
def logout():
    session.clear()
    return redirect('/')

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host='0.0.0.0', port=port)
```

Result

The Flask application was successfully developed and deployed, providing a functional login system and a dynamic portal. Authorized users can securely access cloud facts, while unauthorized access is prevented through session validation.

6. Creating a wharehouse Applications in sales Force.com

Aim

To design and implement a Warehouse Management System in Salesforce using Apex Classes and Triggers to manage stock, create orders, and validate stock availability before order insertion.

Procedure

1. Create Custom Object Product__c with fields Name and Stock_Quantity__c (Number).

2. Create Custom Object Order__c with fields Product_Name__c (Text) and Quantity__c (Number).

3. Create Apex Class WarehouseManagement with methods updateStock() and createOrder().

4. Create Trigger StockValidation on Order__c (before insert) to validate stock.

5. Use Execute Anonymous Window to insert sample records and test functionality.

Source Code

Apex Class: WarehouseManagement

```
public class WarehouseManagement {
   public static void updateStock(String productName, Integer quantity) {
      Product__c product = [
         SELECT Id, Stock_Quantity__c
         FROM Product__c
         WHERE Name = :productName
         LIMIT 1
      ];
      product.Stock_Quantity__c += quantity;
      update product;
   }
   public static void createOrder(String productName, Integer quantity) {
      Product__c product = [
         SELECT Id, Stock_Quantity__c
         FROM Product__c
         WHERE Name = :productName
         LIMIT 1
      ];
      if (product.Stock_Quantity__c >= quantity) {
         product.Stock_Quantity__c -= quantity;
         update product;
         Order__c newOrder = new Order__c(
            Product_Name__c = productName,
            Quantity__c = quantity
```

```
            );
            insert newOrder;
        } else {
            System.debug('Insufficient stock');
        }
    }
}
```

Trigger: StockValidation

```
trigger StockValidation on Order__c (before insert) {
    for (Order__c ord : Trigger.new) {
        Product__c product = [
            SELECT Stock_Quantity__c
            FROM Product__c
            WHERE Name = :ord.Product_Name__c
            LIMIT 1
        ];
        if (ord.Quantity__c > product.Stock_Quantity__c) {
            ord.addError('Not enough stock available');
        }
    }
}
```

Result

The Warehouse Management System was successfully implemented. Stock is updated correctly, orders are validated, and insufficient stock transactions are prevented using Apex class and trigger.

Left side

Output

1. Product records inserted successfully.
2. Query shows Laptop (50), Mobile (30), Tablet (20).
3. When attempting to create order for 100 Laptops,
   Debug Output: 'Insufficient stock'
4. Order insertion prevented when stock is insufficient.

7.Creating an Application in Salesforce.com using Apex programming languages

Aim

To create a custom application in Salesforce.com using Apex Class and Trigger to automate order creation and validate business rules.

Procedure

Step 1: Create Custom Object Order__c with fields Product_Name__c (Text) and Quantity__c (Number).

Step 2: Open Developer Console and create an Apex Class named OrderManagement.

Step 3: Write logic inside the class to insert order records.

Step 4: Create an Apex Trigger named OrderTrigger on Order__c (before insert).

Step 5: Write validation logic to ensure quantity is greater than zero.

Step 6: Use Execute Anonymous Window to run and test the program.

Step 7: Verify inserted records in the Order object.

Source Code

Apex Class: OrderManagement

```
public class OrderManagement {
   public static void createOrder(String productName, Integer quantity) {
      Order__c newOrder = new Order__c(
         Product_Name__c = productName,
         Quantity__c = quantity
      );
      insert newOrder;
   }
}
```

Apex Trigger: OrderTrigger

```
trigger OrderTrigger on Order__c (before insert) {
   for (Order__c order : Trigger.new) {
      if (order.Quantity__c <= 0) {
         order.addError('Quantity must be greater than zero');
      }
   }
}
```

Execution

Run the following code in Execute Anonymous Window:

```
OrderManagement.createOrder('Laptop', 5);
```

Result

The Salesforce application was successfully created using Apex Class and Trigger. The system validates order quantity before insertion and ensures data integrity.

Left side

Output

Case 1: Valid Quantity → Order inserted successfully and visible in Order object.

Case 2: Invalid Quantity (0 or negative) → Error: 'Quantity must be greater than zero' and record not inserted.

8.HDFS File Operations in Apache Hadoop

Aim

To create a directory in Hadoop Distributed File System (HDFS) and upload a file from the local system into HDFS using Hadoop file system commands.

Requirements

- Hadoop installed and configured
- Java installed
- Command Prompt / Terminal
- Python file (basic_cloud.py)

Procedure

1. Install and configure Hadoop in the system.
2. Open **Command Prompt**.
3. Start Hadoop services.
4. Create a directory in HDFS.
5. Upload the Python file into the HDFS directory.
6. Verify whether the file is uploaded successfully.

Commands Used

Create Directory in HDFS

hadoop fs -mkdir /asker

Upload File to HDFS

hadoop fs -put E:\Shasath\basic_cloud.py /asker

List Files in Directory

hadoop fs -ls /asker

Program Code

basic_cloud.py

print("Cloud Computing Lab - Hadoop HDFS Demo")

print("File successfully stored in HDFS")

Input

Local file used as input:

E:\Shasath\basic_cloud.py

Uploaded to HDFS directory:

/asker

Result

The directory **/asker** was successfully created in HDFS and the file **basic_cloud.py** was uploaded successfully using Hadoop file system commands.

Left side

Output

After executing the command:

hadoop fs -ls /asker

Example output:

Found 1 items

-rw-r--r-- 1 user supergroup 90 /asker/basic_cloud.py

9. implementation of para- virtualization using Virtual Box Creation and Execute Linux Commands

Aim

To create a Virtual Machine using VirtualBox and execute basic Linux terminal commands.

Procedure

1. Install VirtualBox on the system.

2. Download the Ubuntu ISO image file.

3. Open VirtualBox and click New to create a new virtual machine.

4. Enter the VM name and select:

Type: Linux

Version: Ubuntu (64-bit).

Allocate RAM (minimum 2048 MB).

Create a virtual hard disk:

Select VDI (VirtualBox Disk Image)

Choose Dynamically Allocated

Set disk size to 20 GB.

5. Go to Settings → Storage and attach the Ubuntu ISO file.

6. Start the virtual machine and install Ubuntu.

7. After installation, open the Terminal.

8. Execute basic Linux commands.

PseudoCode:

Commands used in the terminal (Linux) :

1.pwd: Displays the present working directory.

Command:

pwd

2. ls: Lists files and directories.

Command:

ls

#3. mkdir : Creates a new directory.

Command:

mkdir lab

ls

4.cd: Changes the current directory.

Command:

cd lab

pwd

#5. touch: Creates new empty files.

Command:

touch file1.txt file2.txt

ls

#6. cat: Displays the content of a file.

Command:

cat file1.txt

7. vi: Opens a file in the vi editor for editing.

Command:

vi file1.txt

Press: i

Type text:

"Linux Lab Exercise

Using vi ediotr"

Press : Esc

Save file:

:w

Quit:

:q

Or save and quit:

:wq

#8. cat (after editing)

Command:

cat file1.txt

9. rm: Deletes a file.

Command:

rm file2.txt

ls

#10. clear: Clears the terminal screen.

Command:

clear

Result

The Virtual Machine was successfully created using VirtualBox and basic Linux commands were executed in the Ubuntu terminal.

Left side:

Output:

#1

/home/user

2#

Documents Downloads Pictures

#3

Documents Downloads lab/

#4

/home/user/lab

#5 file1.txt

file2.txt

#6

(empty file)

#7

"file1.txt" 2L, 30C written

#8

Linux Lab Exercise

Using vi editor

#9

file1.txt

#10

Terminal screen cleared

10.Serverless User Feedback Portal using AWS (CASE STUDY)

AIM

To develop a serverless web application that captures user-submitted feedback through a lightweight frontend and securely stores the data in an **Amazon DynamoDB** table using **AWS Lambda**.

PROCEDURE

1. **Database Creation:** Created an **Amazon DynamoDB** table named UserFeedback with a primary key FeedbackID to store the incoming records.
2. **Logic Implementation:** Authored an **AWS Lambda** function in **Python** using the boto3 library to parse the JSON body and insert items into the DynamoDB table.
3. **Permission Management:** Configured an **IAM Role** with DynamoDB write permissions and attached it to the Lambda function to authorize data storage.
4. **API Gateway Setup:** Created a REST API with a /submit resource and a **POST** method to serve as the communication bridge.
5. **Integration Configuration:** Enabled **Lambda Proxy Integration** and **CORS** in API Gateway to allow the frontend to send structured events to Lambda.
6. **Frontend Connection:** Built a minimal **HTML form** with JavaScript fetch() to send user data (Name, Email, Message) to the API's Invoke URL.

PROGRAM

```
import json

import boto3

import uuid


# Initializing DynamoDB resource

dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table('UserFeedback')


def lambda_handler(event, context):

    try:

        # Parsing data from the API Gateway event body

        body = json.loads(event['body'])


        # Inserting item into DynamoDB

        table.put_item(

            Item={

                'FeedbackID': str(uuid.uuid4()),
```

```
            'UserName': body.get('name'),
            'Email': body.get('email'),
            'Message': body.get('message')
        }
    )

    return {
        'statusCode': 200,
        'headers': {
            'Access-Control-Allow-Origin': '*',
            'Content-Type': 'application/json'
        },
        'body': json.dumps({'message': 'Success! Data sent to DynamoDB.'})
    }
  except Exception as e:
    return {
        'statusCode': 500,
        'body': json.dumps({'error': str(e)})
    }
```

///HTML(INDEX – Feedback)

```
<form id="f">
  <input id="n" placeholder="Name" required>
  <input id="e" placeholder="Email" required>
  <textarea id="m" placeholder="Feedback"></textarea>
  <button type="button" onclick="p()">Submit to AWS</button>
</form>

<script>
  async function p() {
    const res = await fetch("https://yec9ocwc47.execute-api.us-east-1.amazonaws.com/prod/submit", {
      method: 'POST',
      body: JSON.stringify({ name: f.n.value, email: f.e.value, message: f.m.value })
```

```
  });
  if (res.ok) alert("Success! Data sent to DynamoDB.");
 }
</script>
```

RESULT

The end-to-end serverless pipeline was successfully implemented and validated through real-time data submission. The system automatically processes the user input and generates a unique entry in the database, confirming a successful cloud integration.

Left side:

OUTPUT

- **User Experience:** Upon clicking the "Submit to AWS" button, the browser displays an alert message: **"Success! Data sent to DynamoDB."**.
- **Database Verification:** The **DynamoDB Console** reflects the stored data with **"Items returned: 1"** and shows the corresponding Name, Email, and Message.